ADA — Automated Data Analyst

Technical Interview Mastery & Live Demo Script Guide | Author: **Kartikeya Mishra** (KartikeyaM2007)

Portfolio: https://port-folio-main-window.vercel.app/ | **Live App**: https://automated-data-analyst-ada.streamlit.app/

SECTION 1: THE 60-SECOND & 3-MINUTE INTERVIEW PITCH

The 60-Second Elevator Pitch (What is ADA?):

"ADA (Automated Data Analyst) is a production-ready, zero-configuration AI Business Intelligence application built with Python, Streamlit, DuckDB, Pandas, and Plotly. Unlike traditional BI tools like Tableau or PowerBI that require manual column mapping and chart building, ADA lets users drop single or multiple CSV/Excel files and instantly generates an automated executive operating view, evidence-backed strategic narratives using Groq LLaMA 3.3 70B, natural language querying (Ask ADA), an interactive SQL Console, and custom chart generation. It maintains a 100% Privacy-First Hybrid Architecture where raw data calculations run locally and only schema metadata is shared with LLMs."

The 3-Minute Technical Architecture Pitch:

"From an architectural standpoint, ADA is structured into three distinct layers:
1. Deterministic Calculation Core: Ingests tabular data via file_io.py, performs cleanings in analysis.py, detects schema roles (Date, Primary Metric, Dimensions) in business_insights.py, detects anomalies in anomalies.py, and forecasts trends in forecasting.py.
2. Relational DuckDB & NLQ Engine: Registers single or multi-CSV datasets into an in-memory **DuckDB** OLAP database (sql_engine.py). User questions in nlq.py are parsed into explicit, auditable QueryPlan objects executed locally with ANSI SQL generation.
3. Guarded AI Synthesis Layer: In ai_insights.py, ADA connects to Groq API (LLaMA 3.3 70B) or OpenAI using Pydantic structured output validation. Raw data rows never leave the session — only calculated aggregate evidence cards enter the prompt to generate decision-ready actions."

SECTION 2: CODE WALKTHROUGH & EXACT LINE REFERENCES

Use these exact file names and line ranges when showing code during an interview:

File Name & Path	Line Range	Core Responsibility & Key Methods
sql_engine.py	L1 – L109	DuckDB relational table registration & ANSI SQL execution (register_tables, execute_query)
nlq.py	L1 – L619	Conversational Natural Language Query engine & QueryPlan execution (answer_question, to_sql)
ai_insights.py	L1 – L340	Groq LLaMA 3.3 70B structured Pydantic response parsing (generate_ai_narrative, plan_query_with_ai)
business_insights.py	L1 – L885	Automated schema detection & driver waterfall calculation (detect_roles, driver_frame)
ui.py	L1 – L477	Streamlit presentation, Custom Chart Studio & evidence cards (render_chart_studio, render_ai_narrative)
app.py	L1 – L647	Main Streamlit orchestrator, session state, passcode lock & tab navigation (render_sidebar, data_tab)

SECTION 3: LIVE DEMO SCRIPT (STEP-BY-STEP)

Step 1: Introduction & App Setup

• **Action to perform**: Open https://automated-data-analyst-ada.streamlit.app/

• **What to say**: "Hi! I'm Kartikeya Mishra. Today I'm demonstrating ADA, a zero-config AI BI application. Notice the clean branding, session passcode lock, and portfolio link in the sidebar."

• **Code to show**: ui.py:L37-L52

Step 2: Multi-CSV Relational Ingestion

• **Action to perform**: Select 'E-Commerce Orders & Customers (Multi-CSV Relational)' demo dataset

• **What to say**: "ADA ingests multiple CSV files—sales.csv and customers.csv—and registers them as relational SQL tables in DuckDB automatically."

• **Code to show**: sql_engine.py:L20-L49

Step 3: Groq LLaMA 3.3 70B Strategic Read

• **Action to perform**: In Executive Brief tab, click 'Generate AI strategic read'

• **What to say**: "ADA sends calculated evidence cards—not raw data rows—to Groq LLaMA 3.3 70B to generate executive summaries, high-confidence actions, and watchouts in sub-seconds."

• **Code to show**: ai_insights.py:L80-L125

Step 4: Conversational Analyst (Ask ADA)

• **Action to perform**: In Ask ADA tab, type 'What are top 5 products by revenue?' and expand math drawer

• **What to say**: "Every question becomes a transparent QueryPlan and DuckDB SQL query executed locally with Pandas."

• **Code to show**: nlq.py:L580-L619

Step 5: SQL Console & Relational Query

• **Action to perform**: In SQL Console tab, click 'Run SQL Query'

• **What to say**: "Here you can run any ANSI SQL query across single or multi-table datasets with millisecond execution timing."

• **Code to show**: sql_engine.py:L34-L50

Step 6: Live Dashboard & Custom Chart Studio

• **Action to perform**: In Live dashboard tab, scroll to Custom Chart Studio and pick 'Pie' or 'Scatter'

• **What to say**: "Users can dynamically generate Bar, Line, Pie, Scatter, and Boxplot charts on any column with Plotly."

• **Code to show**: ui.py:L380-L440

Step 7: Data Room & Semantic Search

• **Action to perform**: In Data Room tab, type 'revenue' into Semantic Search box

• **What to say**: "Finally, ADA offers data quality audits, semantic column search, and 1-click markdown and CSV exports."

• **Code to show**: app.py:L612-L628

SECTION 4: 10 CRITICAL INTERVIEW QUESTIONS & MODEL ANSWERS

Q1: Why did you choose DuckDB over SQLite or pure Pandas?

Answer: DuckDB is a columnar, in-memory OLAP database engine optimized for analytical queries (aggregations, joins, window functions). Unlike SQLite (row-oriented OLTP), DuckDB processes millions of rows in milliseconds with vectorized execution. Compared to pure Pandas, DuckDB allows users to write standard ANSI SQL queries and handles multi-table relational joins with zero memory copy overhead.

Q2: How do you guarantee data privacy when calling external LLMs like Groq?

Answer: ADA uses a strict Privacy-First Hybrid Architecture. 100% of raw row data parsing, filtering, aggregation, DuckDB SQL execution, and anomaly detection are performed locally. When calling Groq (LLaMA 3.3 70B) or OpenAI, ADA serializes ONLY high-level schema roles (ColumnRoles) and computed aggregate evidence cards (e.g. 'Revenue decreased 39.1% in Dec 2025'). Raw uploaded rows never leave the user's local session.

Q3: How does your NLQ engine parse user questions without hallucinating invalid columns?

Answer: The NLQ engine relies on a deterministic rule-based parser first (nlq.py), matching normalized column names and synonyms. If the question requires LLM planning (ai_insights.py), the model is constrained by Pydantic structured output validation (AIQueryPlan). The LLM is provided the exact column list and instructed to set answerable=False if a column doesn't exist. Furthermore, all query plans undergo local validation against the DataFrame before execution.

Q4: How does ADA perform anomaly detection and forecasting?

Answer: Anomaly detection (anomalies.py) combines rolling z-scores, Interquartile Range (IQR), and trend decomposition to flag statistical outliers without false positives on noisy data. Forecasting (forecasting.py) applies a guarded baseline model capturing month-of-year seasonality and linear trend extrapolation, bounded by uncertainty bands and backtested MAPE error margins to prevent unrealistic projections.

Q5: How are multi-CSV relational joins handled automatically?

Answer: When multiple CSVs are uploaded, file_io.py parses each file into a separate Pandas DataFrame. sql_engine.py sanitizes table names (sales, customers) and registers them in DuckDB. ADA detects shared key columns (e.g. Customer ID) via identifier heuristics (detect_roles), allowing users to execute ANSI SQL JOIN queries seamlessly in the SQL Console.

Q6: How do you ensure high software engineering quality and testability?

Answer: ADA follows modular architecture with clear separation of concerns: input parsing (file_io), domain logic (business_insights, anomalies, forecasting), execution engines (sql_engine, nlq), LLM adapters (ai_insights), and presentation (ui, app). The codebase maintains 100% test coverage with 76 automated unit tests (tests/) covering every component.

Q7: What model provider did you choose and why?

Answer: I integrated Groq API utilizing LLaMA 3.3 70B (llama-3.3-70b-versatile). Groq's LPU inference engine delivers sub-second response times (hundreds of tokens/sec), providing near-instant strategic insights. Additionally, ADA supports OpenAI models (gpt-4o, gpt-4o-mini) and custom OpenAI-compatible endpoints (Ollama, LM Studio).

Q8: How does ADA handle large CSV uploads efficiently?

Answer: ADA enforces streaming chunk reads and a configurable row analysis limit (default 10,000 rows for real-time responsiveness). Data processing uses DuckDB and Pandas vectorized operations. Heavy computation functions are decorated with @st.cache_data to ensure zero redundant processing during UI re-renders.

Q9: How did you implement authentication and security control?

Answer: Session security is built into app.py via ADA_APP_PASSWORD environment variable / passcode lock. When enabled, users must enter a valid passcode to unlock session data. API keys are maintained strictly in memory per session and never written to disk or logged.

Q10: What would you improve or build next in ADA v2.0?

Answer: ADA already has built-in Semantic Column & Schema Search in the Data Room to search schema profiles and data attributes in real-time. In ADA v2.0, I would expand this into: (1) Multi-modal vector embedding search over external unstructured PDFs/Docs accompanying datasets using LanceDB/FAISS, (2) Real-time cloud database adapters (PostgreSQL, Snowflake, BigQuery), (3) Automated SQL index optimization recommendations, and (4) Exportable interactive HTML/React dashboard bundles.